Formatos SCRUM para Proyectos de Software

Administración de Servicios de Internet | Proyecto: Servidor web con balanceo de carga | Equipo 1

1. Ficha de inicio y visión del producto

Nombre del proyecto	Servidor web con balanceo de carga
Equipo	Fernando De la Rosa Flores, Fernanda García Ortega, Paola Aguilar García, Luisa Segura Cedeño
Product Owner	Fernanda García Ortega
Scrum Master	Fernando De la Rosa Flores
Fecha de inicio	28 de septiembre de 2026
Repositorio GitHub	https://github.com/FGort/asi-load-balancer-web-server
Problema que se desea resolver	Una organización desea mejorar la disponibilidad de su sitio web mediante la distribución eficiente de las peticiones entrantes.
Usuarios/beneficiarios	Organización (cliente) y usuarios finales que accederán al sitio web desde un equipo cliente.
Visión del producto	Garantizar un acceso web de alta disponibilidad y tolerancia a fallos mediante infraestructura virtualizada.
Objetivo general	Implementar dos servidores web y un equipo central que balancee y distribuya las peticiones de red hacia ellos.
Alcance inicial	Despliegue de 3 VMs (VM1: Balanceador, VM2: WEB1, VM3: WEB2). Instalación de Apache, configuración de DNS, configuración del proxy/balanceador y publicación de contenido. Instalación del motor de base de datos en WEB1 y WEB2 y configuración de la replicación espejo (maestro-esclavo, activa-pasiva) entre ambos servidores. Demostración de disponibilidad ante fallas del servidor web y de la base de datos.
Fuera de alcance	Implementación de certificados HTTPS o monitoreo avanzado (salvo solicitud posterior).

2. Product Backlog

ID	Historia de usuario	Prioridad	Valor	Estimación	Dependencias	Criterios de aceptación	Estado
HU01	Como administrador, quiero configurar 3 VMs para establecer la infraestructura base.	Alta	5	3 SP	Ninguna	Las 3 VMs encienden y tienen conectividad.	Terminada

HU02	Como administrador, quiero instalar Apache en WEB1 y WEB2 y publicar contenido.	Alta	5	5 SP	HU01	Apache activo; contenido identificable en WEB1 y WEB2.	Terminada
HU03	Como administrador, quiero configurar el balanceador en VM1 para distribuir peticiones.	Alta	5	8 SP	HU01, HU02	Tráfico redirigido alternadamente a WEB1 y WEB2.	En progreso
HU04	Como administrador, quiero configurar DNS para acceso por dominio.	Media	4	3 SP	HU03	Resolución apunta a IP del balanceador.	Pendiente
HU05	Como cliente, quiero probar la disponibilidad apagando WEB1.	Alta	5	2 SP	HU03	Tráfico fluye a WEB2 sin perder conexión al apagar WEB1.	Pendiente
HU06	Como administrador, quiero instalar el motor de base de datos en WEB1 y WEB2 para almacenar los datos de la aplicación.	Alta	5	5 SP	HU02	Motor de BD instalado y activo en WEB1 y WEB2; acceso local verificado.	En progreso
HU07	Como administrador, quiero configurar la replicación espejo (maestro-esclavo) entre WEB1 y WEB2 para mantener los datos sincronizados.	Alta	5	8 SP	HU06	Los cambios en la BD maestra se replican en la BD esclava; estado de replicación sin errores.	Pendiente
HU08	Como cliente, quiero probar el failover apagando la BD principal	Alta	5	5 SP	HU07, HU03	Al caer la BD principal, la aplicación sigue operando con la BD	Pendiente

	para verificar la continuidad del servicio.					secundaria; el balanceador no muestra errores 500/502.	

Avance del Sprint: 8 SP terminados de 31 SP comprometidos (HU01 + HU02); 13 SP en progreso (HU03 + HU06).

3. Historia de usuario detallada (Ejemplo Core)

ID / Título	HU03 / Configurar el balanceador o proxy
Como	Administrador de red
Quiero	Configurar el servicio de balanceo de carga (proxy) en la máquina VM1.
Para	Distribuir las peticiones entrantes equitativamente entre los servidores WEB1 y WEB2.
Prioridad	Alta
Estimación / Story Points	8 SP
Dependencias	HU01 (VMs creadas con IP) y HU02 (Apache corriendo).
Criterios de aceptación	1. Proxy instalado en VM1. 2. Reglas apuntan a VM2 y VM3. 3. Acceso exitoso a través de IP del balanceador.
Avance actual	Nginx instalado en VM1; bloque upstream configurado con WEB1 y WEB2. Pendiente: verificar el reparto alternado (round-robin) desde el cliente.
Notas	Se optó por Nginx sobre HAProxy por su configuración más simple del upstream en el entorno de pruebas.

4. Sprint Planning / Sprint Backlog

Sprint		Sprint 1		
Objetivo del Sprint		Infraestructura base instalada, servidores web respondiendo y base de datos replicada (espejo) operativa, con repositorio activo.		
Fecha inicio/fin		28/09/2026 - 05/10/2026		
Integrantes		Fernanda García Ortega, Paola Aguilar García, Luisa Segura Cedeño, Fernando De la Rosa Flores		
ID HU	**Tarea**	**Responsable**	**Est. (h)**	**Estado**
Global	Configurar repositorio de GitHub (asi-load-balancer-web-server)	Fernanda G.	1 h	Hecho
HU01	Instalar S.O. y asignar IPs estáticas en VM1, VM2, VM3.	Paola A.	3.5 h	Hecho
HU02	Instalar servicio Apache en VM2 (WEB1) y VM3 (WEB2).	Luisa S.	1 h	Hecho
HU02	Crear y subir index.html con identificador a WEB1 y WEB2.	Fernanda G.	1 h	Hecho

HU03	Instalar software balanceador en VM1.	Fernando D.	2 h	En progreso
HU06	Instalar y activar el motor de base de datos (MariaDB/MySQL) en VM2 (WEB1) y VM3 (WEB2).	Paola A.	3 h	En progreso
HU07	Configurar la replicación maestro-esclavo (espejo) entre las BD de WEB1 y WEB2 y verificar la sincronización.	Luisa S.	4 h	Pendiente
HU08	Probar el failover: apagar la BD principal y comprobar continuidad del servicio desde el cliente.	Fernando D.	2 h	Pendiente

5. Daily Scrum

Fecha	¿Qué hice ayer?	¿Qué haré hoy?	Impedimentos
28/09/2026	Se creó el repositorio en GitHub y se invitó a los colaboradores.	Instalar el S.O. y asignar IPs estáticas a VM1, VM2 y VM3.	Ninguno
30/09/2026	IPs estáticas configuradas y conectividad verificada entre las 3 VMs.	Instalar Apache en WEB1/WEB2 y publicar el index.html identificable.	Conflicto de IP en VM3 (resuelto).
02/10/2026	Apache activo en WEB1 y WEB2 con contenido identificable; index publicado.	Instalar el balanceador (Nginx) en VM1 y el motor de BD en WEB1/WEB2.	Diferencia de versión de MariaDB entre nodos.

6. Registro de impedimentos

ID	Fecha	Impedimento	Impacto	Responsable	Estado
IMP01	30/09/2026	Conflicto de dirección IP estática de VM3 con otra máquina del host.	Medio	Paola A.	Cerrado
IMP02	02/10/2026	Las versiones de MariaDB en los repositorios de WEB1 y WEB2 difieren; riesgo para la replicación espejo.	Medio	Luisa S.	Abierto

7. Sprint Review

Sprint	Sprint 1
Estado	Se realizará al cierre del Sprint (05/10/2026). Registro de avance a media iteración:
Incremento parcial	Las 3 VMs operativas con IPs estáticas; Apache sirviendo contenido identificable en WEB1 y WEB2; balanceador Nginx y motores de BD en instalación.
Participantes / interesados	Equipo de desarrollo, Profesor titular
Comentarios recibidos	(A completar durante la ceremonia de Sprint Review.)
Historias aceptadas	(A completar durante la ceremonia de Sprint Review.)
Historias no aceptadas y motivo	(A completar durante la ceremonia de Sprint Review.)
Nuevos elementos para Backlog	(A completar durante la ceremonia de Sprint Review.)

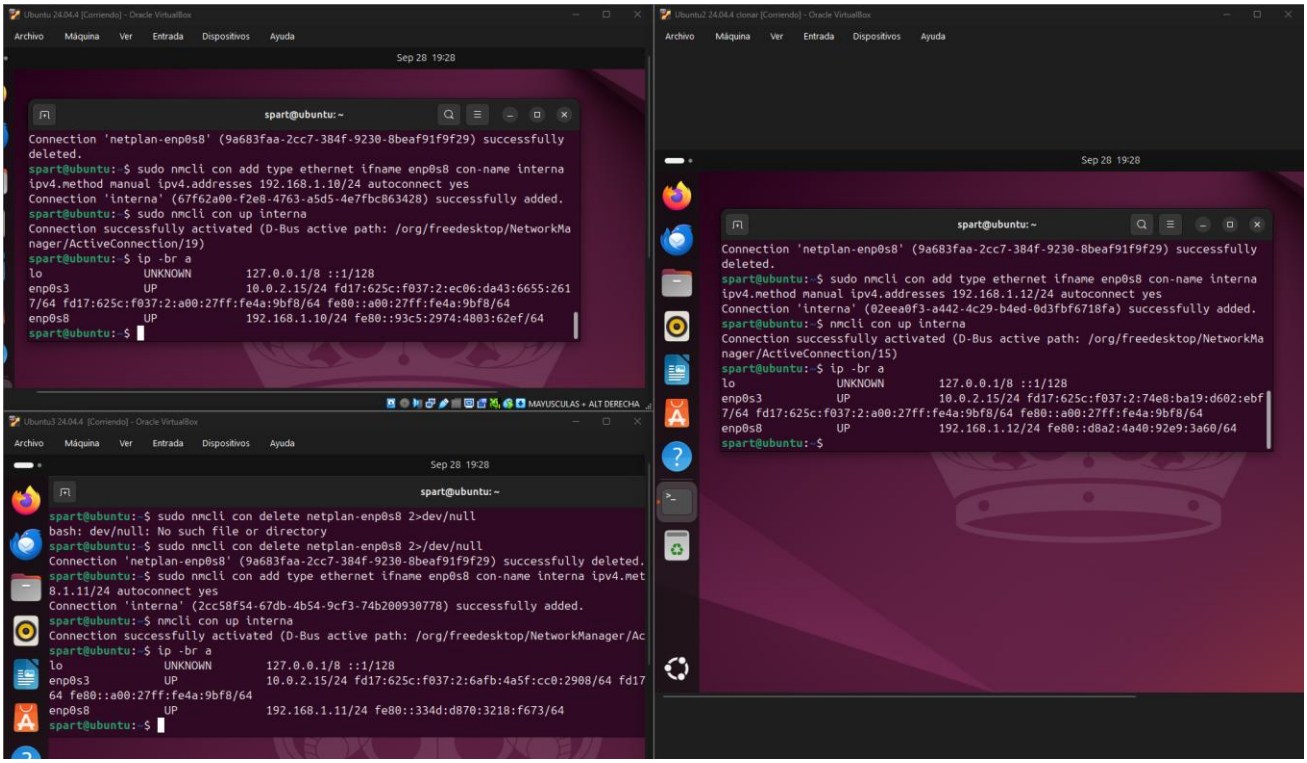
8. Sprint Retrospective

Sprint / Fecha	Sprint 1 / 05/10/2026 (al cierre del Sprint)
¿Qué funcionó bien?	(A completar durante la ceremonia de Sprint Retrospective.)
¿Qué no funcionó?	(A completar durante la ceremonia de Sprint Retrospective.)
¿Qué debemos mejorar?	(A completar durante la ceremonia de Sprint Retrospective.)
Acciones para siguiente Sprint	(A completar durante la ceremonia de Sprint Retrospective.)

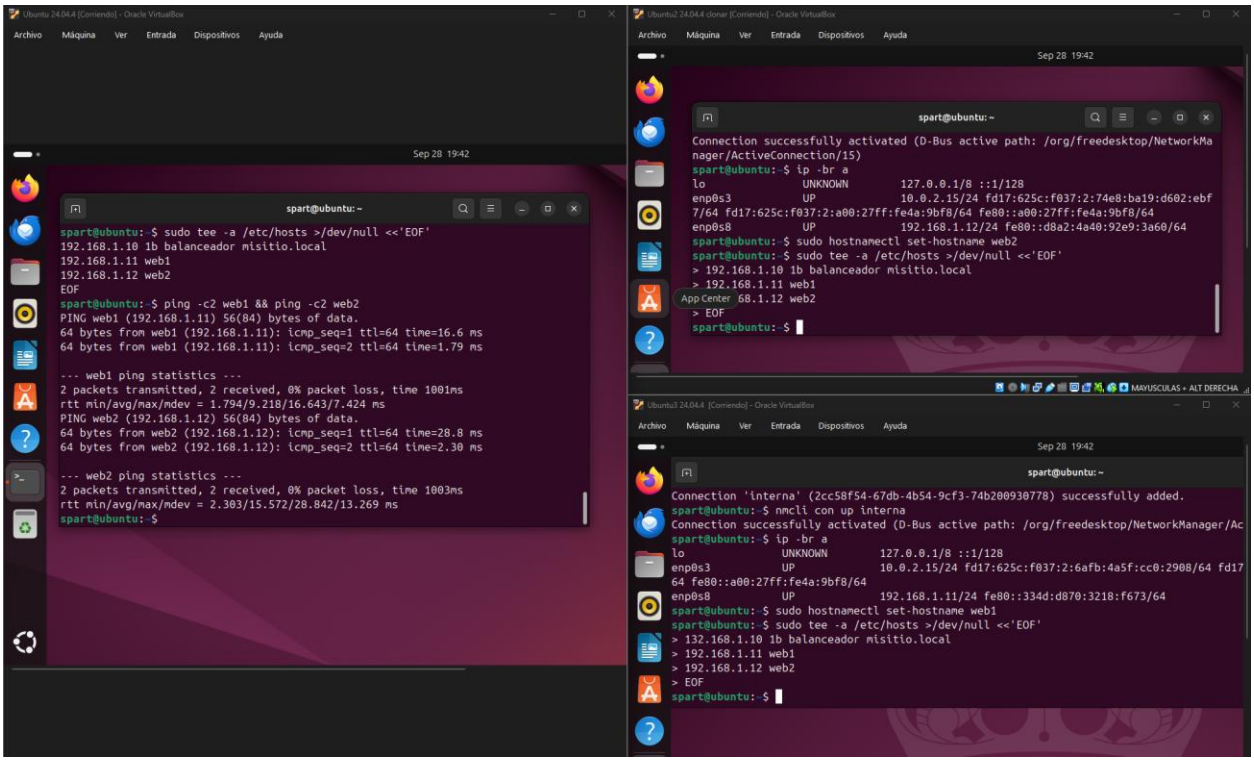
9. Definition of Done (DoD)

Criterio	Sí / No	Evidencia
Código subido al repositorio	Sí	Scripts de configuración de VMs y de Apache versionados en el repositorio de GitHub.
Infraestructura funcional	Parcial	Capturas de las 3 VMs encendidas y de Apache respondiendo en WEB1/WEB2; balanceador y BD en progreso.
Pruebas realizadas	No	Pendiente la prueba de caída de WEB1 y el failover de BD (al terminar HU03/HU07).
Revisión del equipo completada	Parcial	Revisión de pares de las tareas HU01 y HU02 en Daily Scrum.
Documentación actualizada	Sí	Este reporte semanal 3 completado y validado.
El balanceador no muestra errores 500/502 durante la prueba de caída.	No	Se verificará al finalizar la configuración del balanceador (HU03).
Replicación de base de datos verificada.	No	Pendiente: depende de HU06 (motores instalados) y HU07 (replicación).
Configuraciones de base de datos documentadas.	Parcial	Instalación de MariaDB documentada; configuración de replicación pendiente.

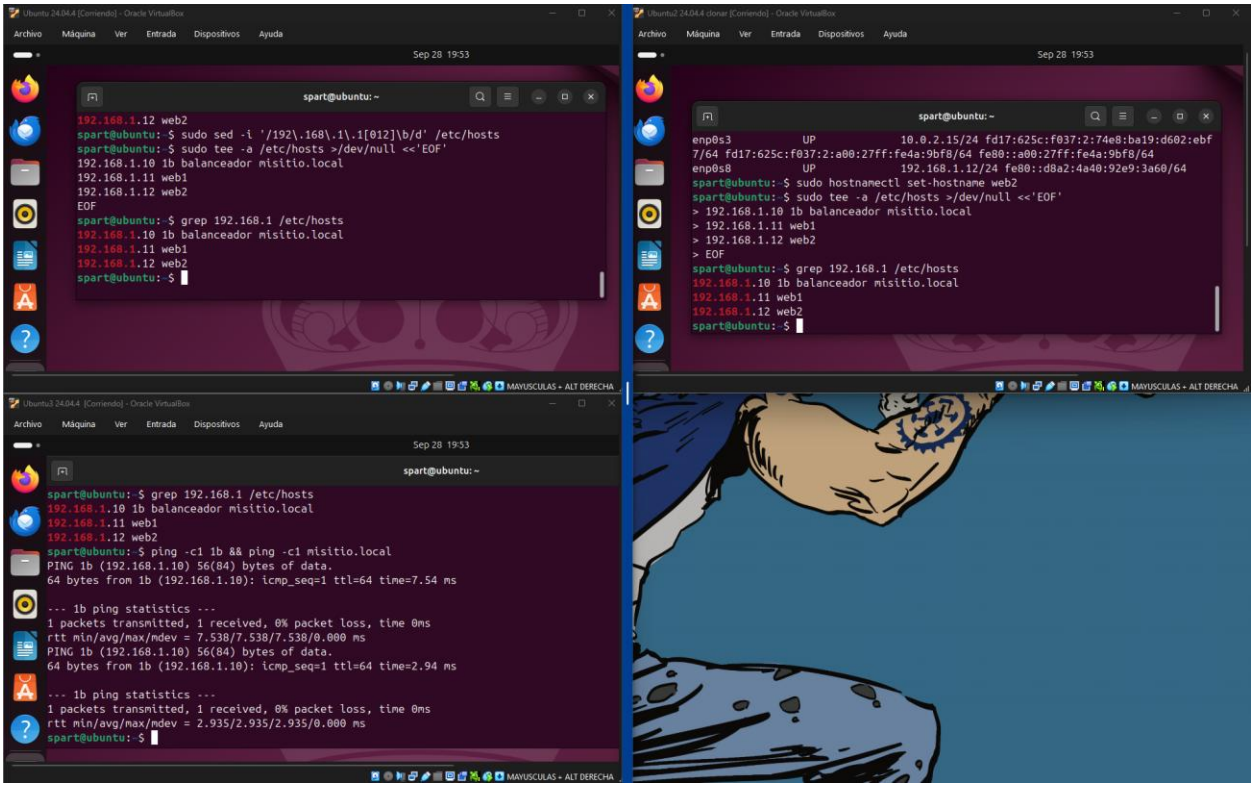
Capturas de Evidencias



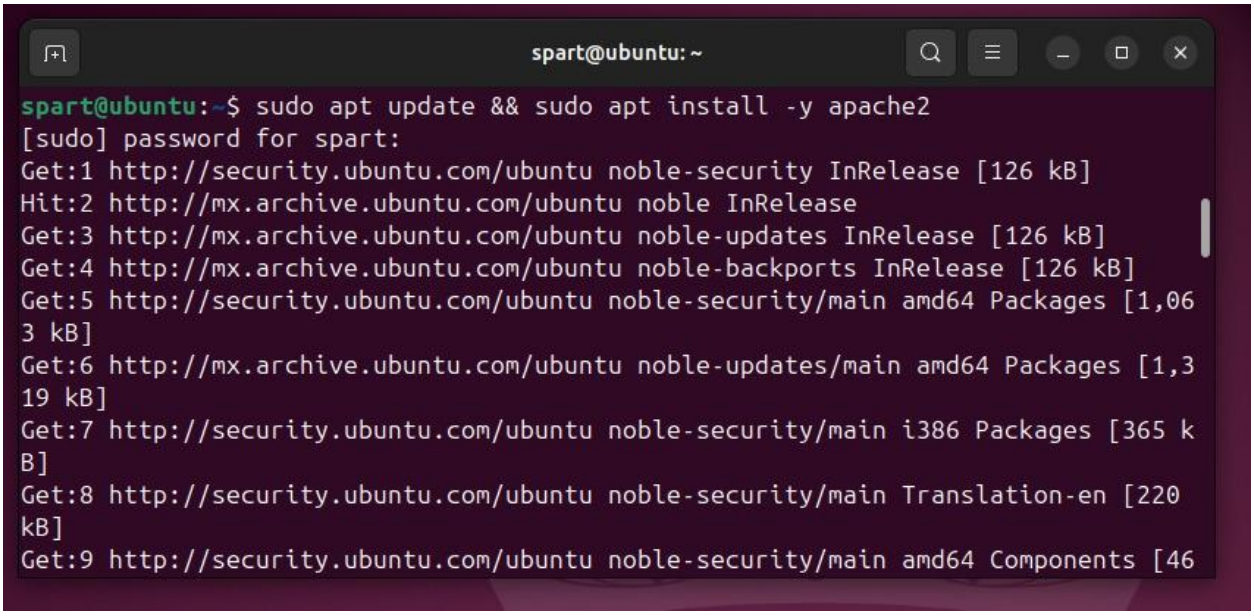
Aquí se ven las terminales donde se crea la conexión "interna" con nmcli en la interfaz enp0s8. Se asignan las IPs estáticas 192.168.1.10, .11 y .12 (/24), se activa la conexión y se verifica con ip -br a.



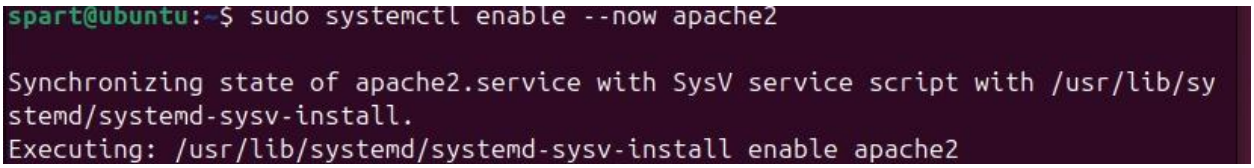
En esta captura podemos ver tres VMs configurando la resolución de nombres. En la primera se agrega al /etc/hosts con tee el balanceador (1b, misitio.local), web1 y web2, y en las otras dos se asigna el hostname (web1 y web2) con hostnamectl y se edita también el /etc/hosts. Desde la primera, el ping por nombre a web1 y web2 responde con 0% de pérdida.



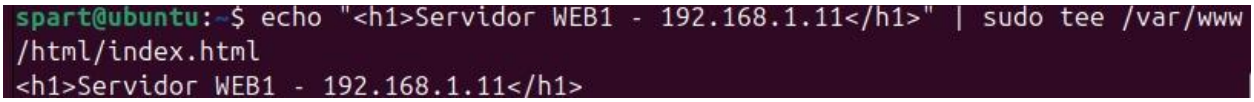
Finalmente en tres VMs se limpian las entradas previas del /etc/hosts con sed, se reescriben y se verifican con grep 192.168.1 /etc/hosts. Desde una VM el ping a 1b y a misitio.local responde correctamente con 0% de pérdida.



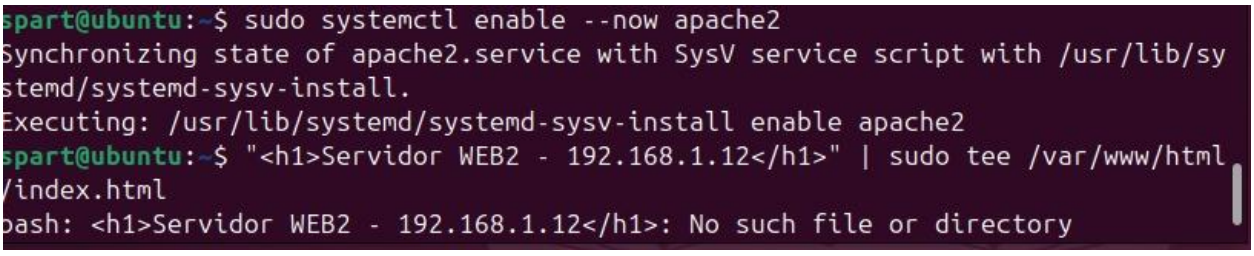
En esta captura se ejecuta sudo apt update && sudo apt install -y apache2 para actualizar la lista de paquetes desde los repositorios de Ubuntu e instalar el servidor web Apache.



Aquí se ejecuta sudo systemctl enable --now apache2, que habilita el servicio de Apache para que arranque con el sistema y lo inicia en ese momento.



En esta terminal se crea el archivo index.html en /var/www/html con echo y sudo tee, con el contenido "Servidor WEB1 - 192.168.1.11" para identificar al servidor.



Aquí se habilita e inicia Apache en WEB2 con systemctl enable --now apache2. Después se intenta crear el index.html, pero se omite el comando echo, por lo que bash interpreta el texto como un comando y muestra el error "No such file or directory".

```
spart@ubuntu:~$ curl localhost
<h1>Servidor WEB1 - 192.168.1.11</h1>
```

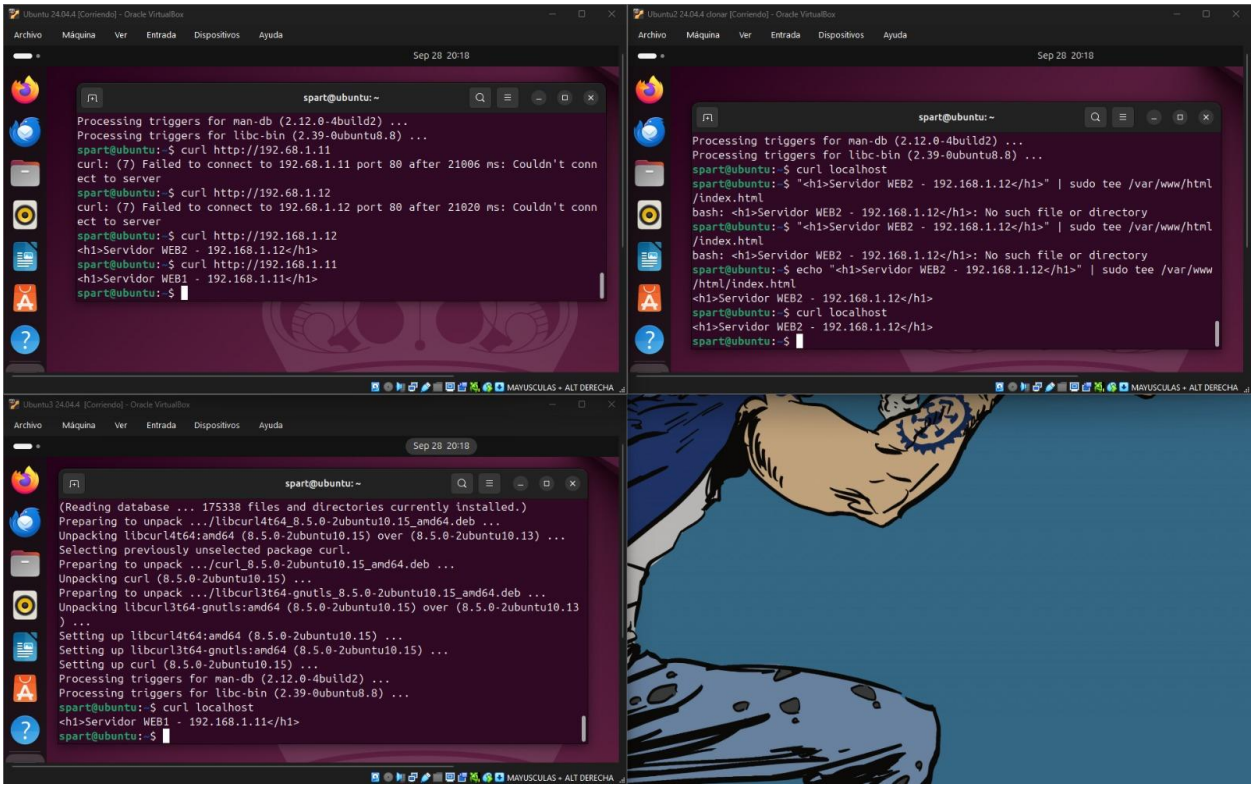
Con curl localhost se comprueba que Apache en WEB1 responde de forma local con la página "Servidor WEB1 - 192.168.1.11".

```
spart@ubuntu:~$ echo "<h1>Servidor WEB2 - 192.168.1.12</h1>" | sudo tee /var/www
/html/index.html
<h1>Servidor WEB2 - 192.168.1.12</h1>
spart@ubuntu:~$ curl localhost
<h1>Servidor WEB2 - 192.168.1.12</h1>
```

Se vuelve a ejecutar el comando, ahora con echo, y se crea correctamente el index.html de WEB2 con el contenido "Servidor WEB2 - 192.168.1.12". Con curl localhost se verifica que el servidor responde con esa página.

```
spart@ubuntu:~$ curl http://192.168.1.12
<h1>Servidor WEB2 - 192.168.1.12</h1>
spart@ubuntu:~$ curl http://192.168.1.11
<h1>Servidor WEB1 - 192.168.1.11</h1>
```

Desde otra VM se ejecuta curl hacia http://192.168.1.12 y http://192.168.1.11, y cada servidor responde con su página identificable, lo que confirma que ambos sitios son accesibles por red.



En esta captura se ven las tres VMs. En la primera, los primeros curl hacia 192.168.1.11 y 192.168.1.12 fallan con "Couldn't connect to server" porque Apache aún no estaba listo, después se repiten y ambos responden correctamente. En la segunda (WEB2) se observan los intentos fallidos sin echo y luego la creación correcta del index.html, verificada con curl localhost. En la tercera (WEB1) se instala curl y se comprueba con curl localhost que responde la página de WEB1.

```
spart@ubuntu:~$ sudo tee /etc/nginx/sites-available/balanceador >/dev/null <<'EOF'
upstream backend_web {
    server 192.168.1.11:80 max_fails=3 fail_timeout=10s;
    server 192.168.1.12:80 max_fails=3 fail_timeout=10s;
}
server {
    listen 80;
    server_name misitio.local;
    location / {
        proxy_pass http://backend_web;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
EOF }   proxy_next_upstream error timeout http_500 http_502 http_503 http_504;
spart@ubuntu:~$ sudo ln -s /etc/nginx/sites-available/balanceador /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
spart@ubuntu:~$ for i in $(seq 1 6); do curl -s http://localhost | grep -o 'WEB[12]'; done
WEB1
WEB2
WEB1
WEB2
WEB1
WEB2
spart@ubuntu:~$
```

En el balanceador se crea el archivo /etc/nginx/sites-available/balanceador con un bloque upstream llamado backend_web que incluye 192.168.1.11:80 y un bloque server que escucha en el puerto 80 para misitio.local. Este bloque redirige las peticiones con proxy_pass, envía los encabezados Host y X-Real-IP, y con proxy_next_upstream pasa al otro servidor si hay errores o tiempos de espera. Después se crea el enlace simbólico en sites-enabled, se elimina el sitio default, se valida la sintaxis con nginx -t y se recarga Nginx. Finalmente, un ciclo for con seis curl a localhost muestra respuestas alternadas WEB1, WEB2, WEB1, WEB2, lo que confirma el reparto round-robin.

```
spart@ubuntu:~$ sudo apt install -y dnsmasq
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  dnsmasq-base
The following NEW packages will be installed:
  dnsmasq
The following packages will be upgraded:
  dnsmasq-base
1 upgraded, 1 newly installed, 0 to remove and 248 not upgraded.
Need to get 399 kB of archives.
After this operation, 97.3 kB of additional disk space will be used.
Get:1 http://mx.archive.ubuntu.com/ubuntu noble-updates/main amd64 dnsmasq-base
 amd64 2.91-0ubuntu0.24.04.1 [381 kB]
Get:2 http://mx.archive.ubuntu.com/ubuntu noble-updates/universe amd64 dnsmasq
all 2.91-0ubuntu0.24.04.1 [17.9 kB]
Fetched 399 kB in 1s (354 kB/s)
(Reading database ... 174686 files and directories currently installed.)
Preparing to unpack .../dnsmasq-base_2.91-0ubuntu0.24.04.1_amd64.deb ...
Unpacking dnsmasq-base (2.91-0ubuntu0.24.04.1) over (2.90-2ubuntu0.4) ...
Selecting previously unselected package dnsmasq.
Preparing to unpack .../dnsmasq_2.91-0ubuntu0.24.04.1_all.deb ...
Unpacking dnsmasq (2.91-0ubuntu0.24.04.1) ...
Setting up dnsmasq-base (2.91-0ubuntu0.24.04.1) ...
```

En esta captura se instala dnsmasq con sudo apt install -y dnsmasq para que el balanceador funcione como servidor DNS local.

```
spart@ubuntu:~$ echo "address=/misitio.local/192.168.1.10" | sudo tee /etc/dnsm
asq.d/misitio.conf
address=/misitio.local/192.168.1.10
spart@ubuntu:~$ sudo systemctl restart dnsmasq
Job for dnsmasq.service failed because the control process exited with error co
de.
See "systemctl status dnsmasq.service" and "journalctl -xeu dnsmasq.service" fo
r details.
```

Aquí se crea el archivo /etc/dnsmasq.d/misitio.conf con la regla address=/misitio.local/192.168.1.10 y se reinicia el servicio. Sin embargo, dnsmasq no logra iniciar y muestra el mensaje "Job for dnsmasq.service failed".

```
spart@ubuntu:~$ sudo tee /etc/dnsmasq.d/misitio.conf >/dev/null <<'EOF'
> listen-address=192.168.1.10
> bind-interfaces
> address=/misitio.local/192.168.1.10
> EOF
spart@ubuntu:~$ sudo systemctl restart dnsmasq
spart@ubuntu:~$ systemctl is-active dnsmasq
active
spart@ubuntu:~$ dig @192.168.1.10 misitio.local +short
192.168.1.10
```

Se reescribe misitio.conf agregando listen-address=192.168.1.10 y bind-interfaces para que dnsmasq escuche solo en la interfaz interna, además de la regla del dominio. Al reiniciar, systemctl is-active devuelve "active", y con dig @192.168.1.10 misitio.local +short se confirma que el dominio resuelve a 192.168.1.10, la IP del balanceador.



Finalmente, en Firefox se accede a http://misitio.local y se muestra la página "Servidor WEB1 - 192.168.1.11", lo que comprueba que el dominio resuelve al balanceador y este redirige la petición a uno de los servidores web.